
DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction

Mohammadreza Pourreza
Department of Computer Science
University of Alberta
Edmonton, CA
pourreza@ualberta.ca

Davood Rafiei
Department of Computer Science
University of Alberta
Edmonton, CA
drafieid@ualberta.ca

Abstract

There is currently a significant gap between the performance of fine-tuned models and prompting approaches using Large Language Models (LLMs) on the challenging task of text-to-SQL, as evaluated on datasets such as Spider. To improve the performance of LLMs in the reasoning process, we study how **decomposing the task into smaller sub-tasks can be effective**. In particular, we show that breaking down the generation problem into sub-problems and feeding the solutions of those sub-problems into LLMs can be an effective approach for significantly improving their performance. Our experiments with three LLMs show that this approach consistently improves their simple few-shot performance by roughly 10%, pushing the accuracy of LLMs towards SOTA or surpassing it. On the holdout test set of Spider, the SOTA, in terms of execution accuracy, was 79.9 and the new SOTA at the time of this writing using our approach is 85.3. Our approach with in-context learning beats many heavily fine-tuned models by at least 5%. Additionally, when evaluated on the BIRD benchmark, our approach achieved an execution accuracy of 55.9%, setting a new SOTA on its holdout test set.

1 Introduction

Natural language interfaces to databases aim at making it easier for end users to access data in a relational database. For example, given the utterance “find employees who make more than their managers” and the schema of tables *employees* and *manages*, one may want to generate a query in SQL that retrieves those employees from a database. Over the past two decades, research in this field has progressed through several phases, with early systems being domain-specific, supporting controlled natural language [Popescu et al., 2003, 2004, Li et al., 2007, Li and Jagadish, 2014] or relying on rule-based approaches [Stratica et al., 2005] while more recent systems offering greater domain-independence using supervised models trained on diverse domains and datasets [Zhong et al., 2017, Yu et al., 2018] and more recently deep neural models trained on large text and code repositories [Dong and Lapata, 2016, Devlin et al., 2018].

The latest development in this progression is the use of Large Language Models (LLMs) under zero-shot and few-shot prompting [Rajkumar et al., 2022, Liu et al., 2023a]. It has been shown that LLMs provide strong baselines using only a few demonstrations and no fine-tuning [Chen et al., 2021, Brown et al., 2020, Liu et al., 2023b]. However, these models fall behind on commonly used benchmarks (e.g., Spider) compared to well-designed and fine-tuned models. Table 1 shows the performance of two latest LLMs, CodeX and GPT-4, on the development set of the Spider dataset. Despite a strong performance, LLMs fall behind, compared to existing methods [Scholak et al., 2021, Li et al., 2023a], especially on medium and complex queries. The question investigated in this paper

is where these LLMs fail and if some of the problems that they are facing can be mitigated to push the performance to reach or surpass fine-tuned SOTA models.

Prompting has several advantages over traditional approaches using pretraining or fine-tuning. The main benefit is that LLMs can perform prediction tasks without requiring large task-specific training data. Training models from scratch or fine-tuning them is a resource-intensive process, often requiring a large number of training samples and machine resources, which may not be available. Additionally, few-shot prompting has been shown to outperform previous state-of-the-art methods on several benchmark datasets and can achieve high accuracy even with limited training examples [Brown et al., 2020, Wei et al., 2022b].

It has been recently shown that the performance of LLMs can be improved on more complex tasks (e.g., math word problems, compositional navigation steps) using approaches such as chain-of-thought [Wei et al., 2022b], least-to-most [Zhou et al., 2022], and decomposed [Khot et al., 2022] prompting techniques where a task is broken down into multiple steps and the intermediate results are used to generate a final answer. Unlike algebraic expressions, which consist of clear steps or operations, breaking a complex SQL query can be a more daunting task because of the declarative structure of the language and the complex relationships between query clauses.

In this paper, we propose a novel method based on few-shot prompting that decomposes the task of natural language text to SQL (referred to as text-to-SQL) into multiple sub-tasks. Previous works on text-to-SQL prompting using LLMs are only evaluated in a zero-shot setting [Rajkumar et al., 2022, Liu et al., 2023a]. However, zero-shot prompting only provides a lower bound on the potential power of LLMs for most tasks [Zhang et al., 2022, Kojima et al., 2022, Wei et al., 2022b, 2021, Brown et al., 2020]. We show that our proposed method outperforms the few-shot prompting method by a large margin. We also compare our method with previous approaches on two cross-domain challenging benchmarks, Spider and BIRD. For Spider dataset, we use the two official evaluation metrics of *execution accuracy* and *exact set match accuracy* [Zhong et al., 2020]. We utilize two variants of the CodeX family, namely Davinci and Cushman [Chen et al., 2021], and the GPT-4 model for prompting. On the holdout test set of Spider, our method achieves an execution accuracy of 85.3% and 78.2% respectively using GPT-4 and CodeX Davinci models and an exact set match accuracy of 60% and 57% respectively using the same models. The large gap between the exact match and execution accuracies is due to the few-shot in-context nature of our method. Pretrained and fine-tuned approaches are more likely to generate SQL queries with a higher exact set match accuracy simply because these models have seen many examples during training that follow the composition style of the queries in the test set (queries in both sets are often written by the same people). Before our work, the SOTA on the test set had an execution accuracy of 79.9% [Li et al., 2023a] and an exact set match accuracy of 74% [Li et al., 2023b], and our method sets a new ground in terms of the execution accuracy. On the BIRD benchmark, our approach achieves a new SOTA result, attaining an execution accuracy of 55.9% on the holdout test set and 50.72% on the development set when employing GPT-4. Moreover, using the *valid efficiency score* introduced in this benchmark, our approach outperformed a GPT-4 baseline, demonstrating a 9% improvement on the development set. This highlights the effectiveness of our method.

Our contributions can be summarized as follows: (1) improving the performance of LLM-based text-to-SQL models through task decomposition, (2) introducing adaptive prompting strategies tailored to task complexity, (3) addressing schema linking challenges in the context of prompting, and (4) using LLMs for self correction. To replicate the reported results, visit our GitHub repository¹ for access to the prompts, results, and the code.

Fine-tuning approaches	
Method	Execution accuracy
RED-SQL 3B + NatSQL [Li et al., 2023a]	84.5
T5-3B + PICARD [Scholak et al., 2021]	79.3
Inference-only approaches	
Method	Execution accuracy
Zero-shot GPT-4 (Ours)	64.9
Few-shot GPT-4 (Ours)	67.4
Zero-shot CodeX [Rajkumar et al., 2022]	55.1
Few-shot CodeX (Ours)	61.5

Table 1: Zero-shot and few-shot prompting compared to fine-tuned approaches on the dev set of Spider

¹<https://github.com/MohammadrezaPourreza/Few-shot-NL2SQL-with-prompting>

2 Related Work

Sequence-to-sequence models [Sutskever et al., 2014] have shown great potential in code generation tasks including text-to-SQL. The key idea is to jointly encode a given natural language question and the database schema and leverage a decoder to predict the target SQL.

On the encoder side, learning a representation for the question and the database schema is carried out using bidirectional LSTM in IRNet [Graves and Graves, 2012], convolutional neural networks in RYANSQL [Choi et al., 2021], pretrained language models such as BERT in SQLova [Hwang et al., 2019] and graph neural networks in RATSQ [Wang et al., 2019], SADGA [Cai et al., 2021], and LGESQL [Cao et al., 2021]. Gan et al. [2021] propose an intermediate representation to bridge the gap between the natural language question and SQL statements. There has been also work on tabular language models that encode both tables and text such as TaBERT [Yin et al., 2020], TaPas [Herzig et al., 2020], and Grappa [Yu et al., 2020].

The methods on the decoder side can be categorized into sketch-based slot-filling and generation-based methods [Qin et al., 2022]. Sketch-based methods break the problem into several slot prediction sub-problems and aggregate the predictions for the slots of the SQL query to be generated [Hwang et al., 2019, Xu et al., 2017, Hui et al., 2021]. A drawback of these methods is that they cannot generalize to queries that do not follow the predefined templates. The generation-based methods [Guo et al., 2019, Wang et al., 2019, Cao et al., 2021, Huang et al., 2021] decode the SQL query as an abstract syntax tree.

In contrast to pretrained and fine-tuned models, Rajkumar et al. [2022] and Liu et al. [2023a] conduct an evaluation of the zero-shot prompting capability of LLMs on text-to-SQL using different prompts on the Spider dataset. Prompting techniques have been also used for tasks such as table understanding, table reasoning, and table-to-text generation [Guo et al., 2023, Chen, 2022], and some remarkable results have been reported using LLMs with just a small number of examples given in the prompt.

3 Few-shot Error Analysis

To better understand where LLMs fail under a few-shot setting, we randomly sampled 500 queries from different databases in the training set of the Spider dataset, excluding all databases used in our prompts. We searched for the queries that produced results different than those of gold queries, hence failing the execution accuracy. We manually examined these failures and classified them into six categories as shown in Figure 1 and discussed next.

Schema linking This category contained the largest number of failed queries and included instances where the model failed to identify column names, table names, or entities mentioned in questions. In some cases, the query required an aggregation function, but a matching column name was chosen instead. For instance, the database schema for question “What are the average and maximum capacities for all stadiums?” included a column named “average”, which was selected by the model instead of taking the average of the capacity column.

JOIN This was the second largest category and included queries that needed a JOIN but the model was unable to identify all the tables required or the correct foreign keys to join the tables.

GROUP BY This category included cases where the SQL statement required a GROUP BY clause, but the model either did not recognize the need for grouping or wrong columns were used for grouping the results.

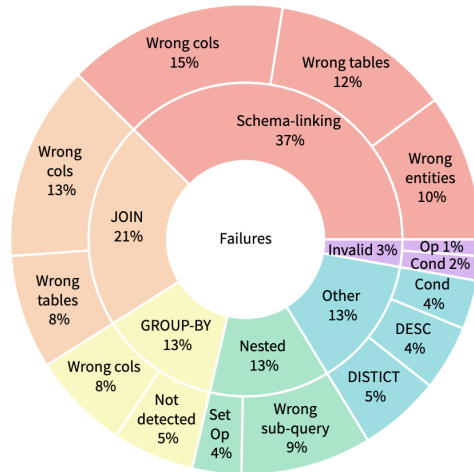


Figure 1: Statistics of simple few-shot failures using CodeX Davinci (Op refers to operators, Cond refers to conditions, and cols refers to columns)

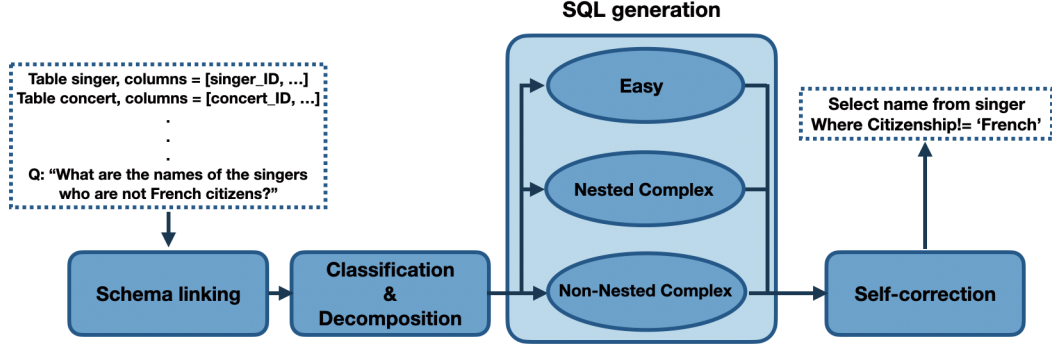


Figure 2: An overview of the proposed methodology including all four modules

Queries with nesting and set operations

For this category, the gold query used nesting or set operations but the model did not recognize the nested structure or was unable to detect the correct nesting or set operation.

Invalid SQL A small set of the generated SQL statements had syntax errors and could not be executed.

Miscellaneous This category included cases that did not fit under any of the previously mentioned categories. Examples included SQL queries that contained extra predicates, missed a predicate, or had missing or redundant DISTINCT or DESC keywords. This category also included cases where the WHERE clause was missing or the query had redundant aggregation functions.

4 Methodology

Despite improvements over zero-shot, few-shot models struggle on more complex queries including those where schema linking is less trivial and the queries that use multiple joins or have a nested structure, as discussed in Section 3.

Our approach to address these challenges is to break down the problem into smaller sub-problems, solve each sub-problem, and use those solutions to construct a solution for the original problem. Similar approaches (e.g., chain-of-thought prompting [Wei et al., 2022b] and least-to-most prompting [Zhou et al., 2022]) have been taken to improve the performance of LLMs on tasks that can be broken down into multiple steps such as math word problems and compositional generalization [Cobbe et al., 2021, Lake and Baroni, 2018]. Unlike these domains where the tasks have a procedural structure with one step directly feeding into the next step, SQL queries in most parts are declarative and the possible steps and their boundaries are less clear. However, the thought process for writing SQL queries may be broken down to (1) detecting database tables and columns that are relevant to the query, (2) identifying the general query structure for more complex queries (e.g., group by, nesting, multiple joins, set operations, etc.), (3) formulating any procedural sub-components if they can be identified, and (4) writing the final query based on the solutions of the sub-problems.

Based on this thought process, our proposed method for decomposing a text-to-SQL task consists of four modules (as depicted in Figure 2): (1) schema linking, (2) query classification and decomposition, (3) SQL generation, and (4) self-correction, which are explained in detail in the following sub-sections. While these modules may be implemented using techniques from the literature, we implement them all using prompting techniques to show that LLMs are capable of solving them all if the problems are simply broken down to the right level of granularity. The few-shot examples used in the prompts are obtained from the training set of the respective benchmarks.

4.1 Schema Linking Module

Schema linking is responsible for identifying references to database schema and condition values in natural language queries. It is shown to help with the generalizability across domains and the synthesis of complex queries [Lei et al., 2020], making it a critical preliminary step in almost all

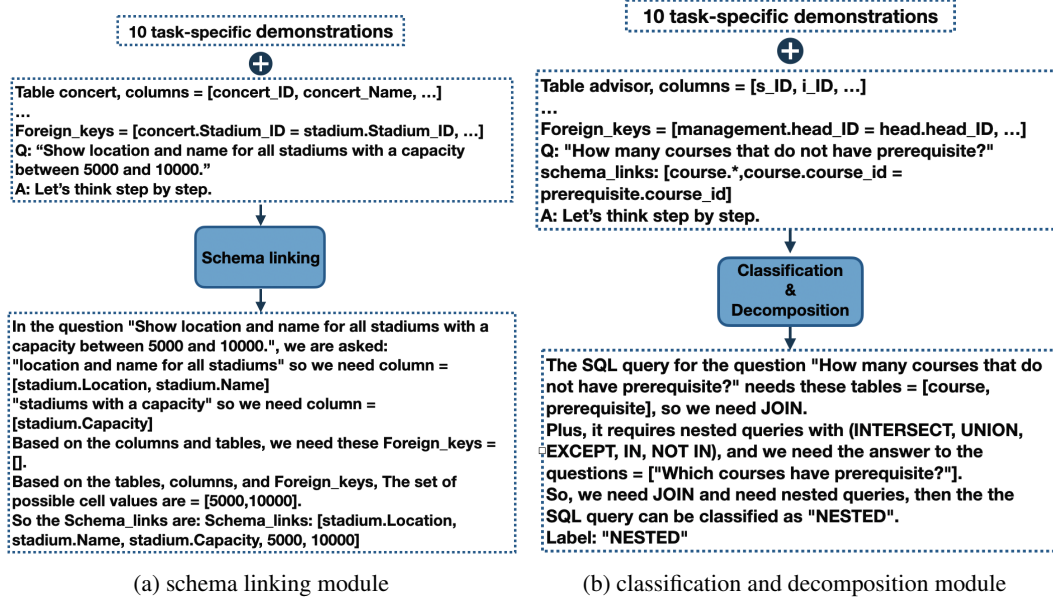


Figure 3: Examples showing the input and output of schema linking (left) and classification and decomposition (right)

existing text-to-SQL methods [Cao et al., 2021, Wang et al., 2019, Guo et al., 2019, Xuan et al., 2021]. This was also a single category with the largest number of failures made by the LLM in our case (Figure 2).

We designed a prompt-based module for schema linking. The prompt includes ten randomly selected samples from the training set of the Spider dataset. Following the chain-of-thought template [Wei et al., 2022b], the prompt begins with "Let's think step by step," as suggested by Kojima et al. [2022]. For each mention of a column name in the question, the corresponding columns and their tables are selected from the given database schema. Possible entities and cell values are also extracted from the question. Figure 3a illustrates an example and the full prompt can be found in Appendix A.3.

4.2 Classification & Decomposition Module

For each join, there is some chance that a correct table or join condition is not detected. As the number of joins in a query increases, the chance that at least one join fails to generate correctly increases. One way to alleviate the problem is introduce a module that detects the tables to be joined. Also some queries have procedural components such as uncorrelated sub-queries, which may be generated independently and be merged with the main query.

To address these issues, we introduce a query classification and decomposition module. The module classifies each query into one of the three classes: easy, non-nested complex and nested complex. The easy class includes single-table queries that can be answered without join or nesting. The non-nested class includes queries that require join but no sub-queries, and the queries in the nested class can contain joins, sub-queries and set operations. The class labels are important for our query generation module, which uses different prompts for each query class. In addition to class labels, query classification and decomposition also detects the set of tables to be joined for both non-nested and nested queries as well as any sub-queries that may be detected for nested queries. Figure 3b shows an example input given to the model and the output that the model generates.

4.3 SQL Generation Module

As the queries become more complex, additional intermediate steps must be incorporated to bridge the gap between the natural language question and the SQL statement. This gap, known as the *mismatch problem* in the literature [Guo et al., 2019], poses a significant challenge to SQL generation, which stems from the fact that SQL is primarily designed for querying relational databases and not

representing the meaning in natural language [Kate, 2008]. While more complex queries can benefit from listing the intermediate steps in a chain-of-thought style prompting, such listings can degrade the performance for simpler tasks [Wei et al., 2022b]. On the same basis, our query generation comprises of three modules, each geared toward different classes.

For questions in our *easy class*, a simple few-shot prompting with no intermediate steps is adequate. The demonstration for an example E_j of this class follows the format $\langle Q_j, S_j, A_j \rangle$, where Q_j and A_j give the query text in English and SQL respectively and S_j indicates the schema links.

Our *non-nested complex* class includes queries that require join. Our error analysis (§ 3) revealed that finding the right columns and foreign keys to join two tables can be challenging for LLMs under simple few-shot prompting, especially when the query requires joining multiple tables. To address this issue, we resort to an intermediate representation to bridge the gap between queries and SQL statements. Various intermediate representations have been introduced in the literature. In particular, SemQL [Guo et al., 2019] removes operators JOIN ON, FROM, and GROUP BY, which have no clear counterparts in natural language queries, and merges the HAVING and WHERE clauses. NatSQL [Gan et al., 2021] builds upon SemQL and removes the set operators. Expressions in natural language queries may not clearly map to a unique SQL clause or they may map to multiple clauses, so removing operators makes the transition from natural language to SQL easier. As our intermediate representation, we use NatSQL, which is shown to have a state-of-the-art performance when combined with other models [Li et al., 2023a]. The demonstration for an example E_j of the non-nested complex class follows the format $\langle Q_j, S_j, I_j, A_j \rangle$, where S_j and I_j respectively denote the schema links and the intermediate representation for the j th example.

Lastly, the nested complex class is the most sophisticated type and requires several intermediate steps before generating the final answer. This class can contain queries that not only require sub-queries using nesting and set operations such as EXCEPT, UNION, and INTERSECT but also multiple table joins, same as the previous class. To break down the problem further into multiple steps, our prompt for this class is designed in a way that the LLM should first solve the sub-queries, generated from the previous module, and then use them to generate the final answer. The prompt for this class follows the format $\langle Q_j, S_j, \langle Q_{j_1}, A_{j_1}, \dots, Q_{j_k}, A_{j_k} \rangle, I_j, A_j \rangle$, where k denotes the number of sub-questions, and Q_{j_i} and A_{j_i} respectively denote the i -th sub-question and the i -th sub-query. As before, Q_j and A_j denote the query in English and SQL respectively, S_j gives the schema links and I_j is a NatSQL intermediate representation.

Full prompts for all three query classes are provided in Appendix A.4, and all examples for the three classes are obtained from the exact same training set database chosen for the classification prompt.

4.4 Self-correction Module

The generated SQL queries can sometimes have missing or redundant keywords such as DESC, DISTINCT and aggregation functions. Our experience with multiple LLMs indicates that these issues are less common in larger LLMs (e.g., queries generated by GPT-4 have less bugs than those from CodeX) but are still present. To address this, we propose a self-correction module where the model is instructed to correct those minor mistakes. This is achieved in a zero-shot setting, where only the buggy code is provided to the model and it is asked to fix the bugs. We propose two different prompts for the self-correction module: *generic* and *gentle*. With a generic prompt, we request the model to identify and correct the errors in the “BUGGY SQL”. The gentle prompt, on the other hand, does not assume the SQL query is buggy, and instead asks the model to check for any potential issues and provides some hints on the clauses to be checked. Our evaluation indicates that a generic prompt can yield a better result with the CodeX model, while a gentle prompt is more effective for the GPT-4 model. Unless explicitly stated otherwise, the default self-correction prompt in DIN-SQL is set to gentle for GPT-4 and generic for CodeX. Examples of both generic and gentle self-correction prompts can be found in Appendix A.6.

5 Experiments

5.1 Models

We evaluated the proposed method using two variants of the CodeX family (Davinci and Cushman variants) and the GPT-4 model. These are the largest open-access LLMs at the time of writing this

paper. Smaller models are less applicable since prompting is believed to be an emergent ability of the LLMs with the number of parameters in the scale of billions [Wei et al., 2022a].

5.2 Hyperparameter

All models were accessed via the OpenAI API. Greedy decoding was used to generate the output by setting the temperature at zero. The max tokens was set to 350 for the self-correction module and 600 for all other modules. The stopping token sequence was set to “#;\n\n” for the self-correction module and “Q:” for all other modules.

5.3 Dataset

Our evaluation was conducted on two cross-domain challenging datasets, Spider and BIRD. Spider consists of 10,181 questions and 5,693 unique complex SQL queries across 200 databases, covering 138 domains, each containing multiple tables. The standard protocol for this dataset divides it into 8,659 training examples across 146 databases, 1,034 development examples across 20 databases, and a holdout of 2,147 test examples across 34 databases. The databases used in each of these sets are non-overlapping. SQL queries are categorized into four difficulty levels, based on the number of SQL keywords used, the presence of nested subqueries, and the usage of column selections and aggregations. BIRD comprises an extensive dataset with 12,751 unique question-SQL pairs, encompassing 95 large databases totaling 33.4 GB in size. It spans a wide array of more than 37 professional domains, including blockchain, hockey, healthcare, and education. BIRD also introduces external knowledge as an additional resource to assist models in generating accurate SQL queries. Specifically four sources of external knowledge were introduced: numeric reasoning knowledge, domain knowledge, synonym knowledge, and value illustration. Notably, the SQL queries in the BIRD dataset tend to be more intricate than those in the Spider dataset. Language models without access to database content often encounter challenges with schema linking. Therefore, our prompts for the BIRD dataset include sample rows from each table to aid the model in schema linking. Furthermore, we have concatenated the provided external knowledge for each question as a hint, placed immediately after each question. However, due to constraints such as limited context window size, the presence of external knowledge, and the inclusion of sample rows, we have had to reduce the number of demonstrations within the prompts for the BIRD dataset.

Model	EX	EM
DIN-SQL + GPT-4 (Ours)	85.3	60
RESDSL-3B + NatSQL (DB content used) [Li et al., 2023a]	79.9	72
DIN-SQL + CodeX davinci (Ours)	78.2	57
Graphix-3B+PICARD (DB content used) [Li et al., 2023b]	77.6	74
SHiP+PICARD (DB content used) [Zhao et al., 2022]	76.6	73.1
N-best Rerankers + PICARD (DB content used) [Zeng et al., 2022]	75.9	72.2
RASAT+PICARD (DB content used) [Qi et al., 2022]	75.5	70.9
T5-3B+PICARD (DB content used) [Scholak et al., 2021]	75.1	71.9
RATSQL+GAP+NatSQL (DB content used) [Gan et al., 2021]	73.3	68.7
RYANSQL v2 + BERT [Choi et al., 2021]	-	60.6
SmBoP + BART [Rubin and Berant, 2020]	-	60.5

Table 2: Execution accuracy (EX) and exact set match accuracy (EM) on the holdout test set of Spider

5.4 Metrics

The performance of our models are evaluated using the official metrics of each dataset: exact-set-match accuracy (EM) and execution accuracy (EX) for Spider and valid efficiency score (VES) and execution accuracy (EX) for BIRD.

The exact-set-match accuracy (EM) treats each clause as a set and compares the prediction for each clause to its corresponding clause in the reference query. A predicted SQL query is considered correct only if all of its components match the ground truth. This metric does not take values into account. The execution accuracy (EX) compares the execution output of the predicted SQL query with that of the ground truth SQL query on some database instances. Execution accuracy provides a more precise estimate of the model’s performance since there may be multiple valid SQL queries

for a given question, and exact set match accuracy only evaluates the predicted SQL against one of them. The Valid Efficiency Score (VES) is a metric designed to measure the efficiency of running the generated SQL queries. This metric is meaningful if the generated queries are correct, meaning their result matches that of the reference query. Therefore, the VES metric takes into account both the accuracy of the generated queries and their efficiency in terms of the execution time.

5.5 Results

5.5.1 Test set results

As shown in Table 2 for the holdout test set of Spider, our method achieves the highest execution accuracy using GPT-4 and the third-highest execution accuracy using CodeX Davinci among all officially published results at the time of this writing. This is achieved without even utilizing the database content. In terms of exact set match accuracy, our method achieves comparable results to previous works that do not utilize database content. As demonstrated in Table 3a, in the case of the BIRD dataset, our method using GPT-4 achieved a test set execution accuracy of 55.9%, setting a new SOTA.

5.5.2 Development set results

Most of our evaluation during development was conducted on the development set of Spider which was easily accessible unlike the test set that was only accessible through an evaluation server provided by Yu et al. [2018]. Table 4a shows the performance of our method using different LLMs, compared to zero-shot prompting of Rajkumar et al. [2022] and Liu et al. [2023a] and our own few-shot prompting. To ensure a fair comparison for the few-shot prompting, we incorporate all the examples utilized for our three classes (easy, non-nested complex, and nested complex) inside the prompt. Given that the CodeX Cushman model has a smaller input context size than the CodeX Davinci and the GPT-4 models, we only use 2 examples from each class (for a total of 6 examples).

Our method significantly outperforms both simple few-shot prompting and zero-shot prompting, in terms of both exact set match and execution accuracies, and the improvement is consistent across all models despite their sizes. For example, compared to few-shot prompting, our method improves the execution accuracy for all models by at least 10%.

On the development set of BIRD, our approach demonstrates a substantial improvement, achieving a 4% gain in execution accuracy and a remarkable 9% improvement in valid efficiency score over a GPT-4 baseline Li et al. [2023c], establishing a new SOTA. These and other results are reported in Table 3b.

The performance of our method on the test set (as reported in Tables 2 and 3a) is higher than that on the development set for both Spider and BIRD. It is hard to pinpoint the exact reason when the test set is hidden, but we speculate that fewer questions in the test set may require the knowledge of the database content, making it easier for our method to predict a correct SQL query. Furthermore, the development set has schema ambiguity (e.g., a query entity can be mapped to multiple database entities but only one is considered correct), and it is possible that the test set has less ambiguity.

We further analyzed the performance of our proposed method on queries with different levels of difficulty. Table 4b presents the performance of our proposed method compared to a basic few-shot prompting on the development set of Spider. Our proposed method outperforms the basic few-shot prompting across all difficulty levels, with the greatest improvement in performance observed for the extra hard and hard classes where the few-shot prompting performed poorly. Our improvement

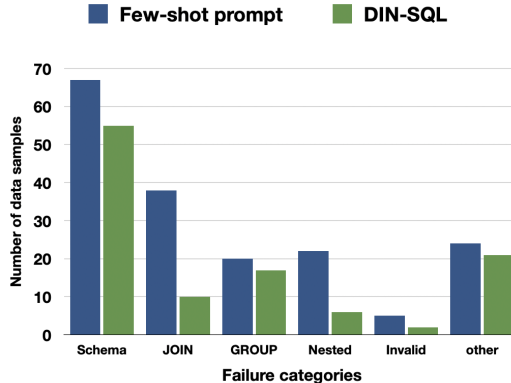


Figure 4: The break-down of failure cases for DIN-SQL (green) and the basic few-shot prompting (blue) across different categories

Model	VES	EX
DIN-SQL + GPT-4 (Ours)	59.44	55.9
GPT-4	60.77	54.89
Claude-2	-	49.02
ChatGPT + CoT [Li et al., 2023c]	56.56	40.08
ChatGPT	51.40	39.30
Codex	41.60	36.47
Palm-2	-	33.04
T5-3B	27.80	24.05
T5-Large	25	20.94
T5-Base	14.7	12.89

(a) Execution accuracy (EX) and Valid Efficiency Score (VES) on the holdout test set of BIRD

Model	VES	EX
DIN-SQL + GPT-4 (Ours)	58.79	50.72
GPT-4	49.77	46.35
Claude-2	-	42.70
ChatGPT + CoT [Li et al., 2023c]	42.30	36.64
ChatGPT	43.81	37.22
Codex	43.41	34.35
Palm-2	-	27.38
T5-3B	25.57	23.34
T5-Large	22.74	19.75
T5-Base	12.90	11.54

(b) Execution accuracy (EX) and Valid Efficiency Score (VES) on the development set of BIRD

Table 3: Performance of DIN-SQL on BIRD development set and test set.

on the easy class (compared to basic few-shot) is due to incorporating schema links in the prompt, highlighting the importance of our schema-linking module.

Prompting	Model	EX	EM
DIN-SQL (Ours)	GPT-4	74.2	60.1
	CodeX Davinci	69.9	57.2
	CodeX Cushman	47.6	35.7
Few-shot (Ours)	GPT-4	67.4	54.3
	CodeX Davinci	61.5	50.2
	CodeX Cushman	43.1	30.9
Zero-shot (Ours)	GPT-4	64.9	40.4
Zero-shot [Liu et al., 2023a]	ChatGPT	60.1	-
Zero-shot [Rajkumar et al., 2022]	CodeX Davinci	47.5	
Zero-shot (DB content used) [Rajkumar et al., 2022]	CodeX Davinci	55.1	
	CodeX Cushman	53	
	GPT3	21.7	

(a) Performance compared to zero-shot and few-shot prompting using different LLMs on the dev set of Spider

Execution accuracy (EX)						
Prompting	Model	Easy	Medium	Hard	Extra	All
DIN-SQL	GPT-4	91.1	79.8	64.9	43.4	74.2
DIN-SQL	CodeX Davinci	89.1	75.6	58	38.6	69.9
Few-shot	GPT-4	86.7	73.1	59.2	31.9	67.4
Few-shot	CodeX Davinci	84.7	67.3	47.1	26.5	61.5
Exact set match accuracy (EM)						
Prompting	Model	Easy	Medium	Hard	Extra	All
DIN-SQL	GPT-4	82.7	65.5	42	30.7	60.1
DIN-SQL	CodeX Davinci	78.6	67.3	38.5	17.5	57.2
Few-shot	GPT-4	87.9	54	47.1	12	54.3
Few-shot	CodeX Davinci	77	53.8	38.5	12.7	50.2

(b) Performance compared to our basic few-shot prompting across different query difficulty levels

Table 4: Performance of DIN-SQL against other in-context learning approaches

5.5.3 Error improvements

In Section 3, we did an error analysis of basic few-shot prompting on 500 queries randomly chosen from the training set. To understand the degree those errors are resolved, we ran DIN-SQL on the same 500 queries. As shown in Figure 4, our proposed approach improves the performance for all categories with the largest improvement seen for the JOIN and Nested categories. Despite having an explicit module for schema-linking, the largest portion of failure cases still belong to this category.

5.6 Ablation study

In an ablation study, we evaluated our approach with and without each of the four modules. As shown in Table 5 for the CodeX Davinci model, excluding any of the modules leads to an overall decrease in performance, in terms of the execution accuracy.

More details emerge as we study the effectiveness of each module across different query classes. Schema linking helps all query classes with the least improvement for the hard class. Our inspection of a sample of the failed cases reveals that schema linking sometimes finds redundant links due to an ambiguity in the question or schema, and this can introduce redundant joins or output columns.

Without a classification, we had to use either a simple few-shot prompting or a decomposed chain-of-thought (COT) prompting for all queries. The reported performance without a classification

Prompting	Model	Easy	Medium	Hard	Extra	All
DIN-SQL (generic self-corr)	CodeX Davinci	89.1	75.6	58	38.6	69.9
DIN-SQL (gentle self-corr)	CodeX Davinci	87.5	76.9	51.7	36.1	68.7
DIN-SQL w/o self-corr	CodeX Davinci	83.9	75.4	52.3	36.1	67.3
DIN-SQL w/o schema-linking	CodeX Davinci	87.3	70.6	57.6	27.1	65.9
DIN-SQL w/o classification (simple few-shot prompting)	CodeX Davinci	87.9	68.2	51.7	27.1	63.1
DIN-SQL w/o classification (decomposed COT prompting)	CodeX Davinci	84.2	71.2	54.3	38.6	68.2
DIN-SQL (gentle self-corr)	GPT-4	91.1	79.8	64.9	43.4	74.2
DIN-SQL (generic self-corr)	GPT-4	89.9	76.5	59.2	34.3	70.0
DIN-SQL w/o self-correc	GPT-4	91.1	79.1	63.2	41.6	73.3

Table 5: Performance of our method, in terms of execution accuracy, on the dev set with and without each module

module in Table 5 is for our comprehensive framework that includes all our components except classification. This means that the approach contains not only COT prompting but also Schema Linking, Self-Correction, and NatSQL Intermediate Representation, all of which are significant contributions of our work. The decomposed chain-of-thought result presented in this table refers to employing the most complex prompt, developed for the nested complex class, for all questions instead of adopting a classification-based approach to determine prompt complexity based on the question’s level of difficulty. In contrast, the result for the DIN-SQL with simple few-shot prompting refers to using the simplest prompting class, easy class, for all questions across different level’s of difficulty. As expected, a decomposed chain-of-thought prompting works better for hard and extra hard queries whereas a simple few-shot works better for the easy class.

For self-correction, we ran our study using both CodeX Davinci and GPT-4. For CodeX Davinci, a generic self-correction prompt helps the model across all query classes. A gentle self-correction prompt is also helpful but the gain is smaller than generic one for CodeX Davinci. However, there is less chance that GPT-4 generates a buggy code, and giving a generic prompt of “Buggy SQL:. . . Fixed SQL:. . .” can hurt the performance. A gentle prompt work better for GPT-4 and improves the performance across all of the classes except the easy class.

6 Conclusions

Prompting has enabled large language models to achieve impressive performance on numerous NLP tasks across different domains, without requiring a large training set. Prior to our research, the effectiveness of prompting methods utilizing LLMs for the text-to-SQL task was inferior to that of models fine-tuned for the task. To bridge this gap, we have devised a decomposition technique to tackle some of the challenges that caused this disparity. Our extensive experiments on two challenging datasets of Spider and BIRD show that our method significantly improves the performance of prompting across all query classes, producing comparable or even superior results to state-of-the-art fine-tuned approaches.

7 Limitations

There are some limitations or areas of improvement to this work. Our manually constructed demonstrations are fixed for each query class. Future research may explore adaptive and automated methods for generating demonstrations at finer granularities, which can further enhance the performance of our approach. Additionally, as of the time of writing this paper, our proposed approach, characterized by its decomposed and step-by-step structure, incurs a cost of approximately \$0.5 and exhibits a latency of approximately 60 seconds when responding to a natural language question from the Spider dataset using GPT-4. We anticipate that as LLMs continue to advance, these costs and latencies should decrease, but reducing the cost is another possible direction.

Acknowledgement

This research was funded by Natural Sciences and Engineering Research Council of Canada. We wish to thank Tao Yu and Hongjin Su for running our code on the hold out test set of Spider and Jinyang Li, Binyuan Hui, Reynold Cheng, Ge Qu and the other authors of BIRD for running our code on the holdout test set of BIRD. We also wish to thank Csaba Czepesvari, Dale Schuurmans and the anonymous reviewers of NeurIPS for their constructive comments to improve this work.

References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Ruichu Cai, Jinjie Yuan, Boyan Xu, and Zhifeng Hao. Sadga: Structure-aware dual graph aggregation network for text-to-sql. *Advances in Neural Information Processing Systems*, 34:7664–7676, 2021.
- Ruisheng Cao, Lu Chen, Zhi Chen, Yanbin Zhao, Su Zhu, and Kai Yu. Lgesql: line graph enhanced text-to-sql model with mixed local and non-local relations. *arXiv preprint arXiv:2106.01093*, 2021.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Wenhu Chen. Large language models are few (1)-shot table reasoners. *arXiv preprint arXiv:2210.06710*, 2022.
- DongHyun Choi, Myeong Cheol Shin, EungGyun Kim, and Dong Ryeol Shin. Ryansql: Recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases. *Computational Linguistics*, 47(2):309–332, 2021.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Li Dong and Mirella Lapata. Language to logical form with neural attention. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33–43, 2016.
- Yujian Gan, Xinyun Chen, Jinxia Xie, Matthew Purver, John R Woodward, John Drake, and Qiaofu Zhang. Natural sql: Making sql easier to infer from natural language specifications. *arXiv preprint arXiv:2109.05153*, 2021.
- Alex Graves and Alex Graves. Long short-term memory. *Supervised sequence labelling with recurrent neural networks*, pages 37–45, 2012.
- Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. Towards complex text-to-sql in cross-domain database with intermediate representation. *arXiv preprint arXiv:1905.08205*, 2019.
- Zhixin Guo, Minyxuan Yan, Jiexing Qi, Jianping Zhou, Ziwei He, Zhouhan Lin, Guan jie Zheng, and Xinbing Wang. Few-shot table-to-text generation with prompt planning and knowledge memorization. *arXiv preprint arXiv:2302.04415*, 2023.
- Jonathan Herzig, Paweł Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. Tapas: Weakly supervised table parsing via pre-training. *arXiv preprint arXiv:2004.02349*, 2020.

- Junyang Huang, Yongbo Wang, Yongliang Wang, Yang Dong, and Yanghua Xiao. Relation aware semi-autoregressive semantic parsing for nl2sql. *arXiv preprint arXiv:2108.00804*, 2021.
- Binyuan Hui, Xiang Shi, Ruiying Geng, Binhua Li, Yongbin Li, Jian Sun, and Xiaodan Zhu. Improving text-to-sql with schema dependency learning. *arXiv preprint arXiv:2103.04399*, 2021.
- Wonseok Hwang, Jinyeong Yim, Seunghyun Park, and Minjoon Seo. A comprehensive exploration on wikisql with table-aware word contextualization. *arXiv preprint arXiv:1902.01069*, 2019.
- Rohit Kate. Transforming meaning representation grammars to improve semantic parsing. In *CoNLL 2008: Proceedings of the Twelfth Conference on Computational Natural Language Learning*, pages 33–40, 2008.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*, 2022.
- Brenden Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International conference on machine learning*, pages 2873–2882. PMLR, 2018.
- Wenqiang Lei, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua. Re-examining the role of schema linking in text-to-sql. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6943–6954, 2020.
- Fei Li and Hosagrahar V Jagadish. Constructing an interactive natural language interface for relational databases. *Proceedings of the VLDB Endowment*, 8(1):73–84, 2014.
- Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. Decoupling the skeleton parsing and schema linking for text-to-sql. *arXiv preprint arXiv:2302.05965*, 2023a.
- Jinyang Li, Binyuan Hui, Reynold Cheng, Bowen Qin, Chenhao Ma, Nan Huo, Fei Huang, Wenyu Du, Luo Si, and Yongbin Li. Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing. *arXiv preprint arXiv:2301.07507*, 2023b.
- Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiayi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Chenhao Ma, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls, 2023c.
- Yunyao Li, Huahai Yang, and HV Jagadish. Nalix: A generic natural language search environment for xml data. *ACM Transactions on database systems (TODS)*, 32(4):30–es, 2007.
- Aiwei Liu, Xuming Hu, Lijie Wen, and Philip S Yu. A comprehensive evaluation of chatgpt’s zero-shot text-to-sql capability. *arXiv preprint arXiv:2303.13547*, 2023a.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023b.
- Ana-Maria Popescu, Oren Etzioni, and Henry Kautz. Towards a theory of natural language interfaces to databases. In *Proceedings of the 8th international conference on Intelligent user interfaces*, pages 149–157, 2003.
- Ana-Maria Popescu, Alex Armanasu, Oren Etzioni, David Ko, and Alexander Yates. Modern natural language interfaces to databases: Composing statistical parsing with semantic tractability. In *COLING 2004: Proceedings of the 20th International Conference on Computational Linguistics*, pages 141–147, 2004.
- Jiexing Qi, Jingyao Tang, Ziwei He, Xiangpeng Wan, Chenghu Zhou, Xinbing Wang, Quanshi Zhang, and Zhouhan Lin. Rasat: Integrating relational structures into pretrained seq2seq model for text-to-sql. *arXiv preprint arXiv:2205.06983*, 2022.

- Bowen Qin, Binyuan Hui, Lihan Wang, Min Yang, Jinyang Li, Binhua Li, Ruiying Geng, Rongyu Cao, Jian Sun, Luo Si, et al. A survey on text-to-sql parsing: Concepts, methods, and future directions. *arXiv preprint arXiv:2208.13629*, 2022.
- Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. Evaluating the text-to-sql capabilities of large language models. *arXiv preprint arXiv:2204.00498*, 2022.
- Ohad Rubin and Jonathan Berant. Smbop: Semi-autoregressive bottom-up semantic parsing. *arXiv preprint arXiv:2010.12412*, 2020.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. *arXiv preprint arXiv:2109.05093*, 2021.
- Niculae Stratica, Leila Kosseim, and Bipin C Desai. Using semantic templates for a natural language interface to the cindi virtual library. *Data & Knowledge Engineering*, 55(1):4–19, 2005.
- Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.
- Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. *arXiv preprint arXiv:1911.04942*, 2019.
- Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022a.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022b.
- Xiaojun Xu, Chang Liu, and Dawn Song. Sqlnet: Generating structured queries from natural language without reinforcement learning. *arXiv preprint arXiv:1711.04436*, 2017.
- Kuan Xuan, Yongbo Wang, Yongliang Wang, Zujie Wen, and Yang Dong. Sead: end-to-end text-to-sql generation with schema-aware denoising. *arXiv preprint arXiv:2105.07911*, 2021.
- Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. Tabert: Pretraining for joint understanding of textual and tabular data. *arXiv preprint arXiv:2005.08314*, 2020.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018.
- Tao Yu, Chien-Sheng Wu, Xi Victoria Lin, Bailin Wang, Yi Chern Tan, Xinyi Yang, Dragomir Radev, Richard Socher, and Caiming Xiong. Grappa: Grammar-augmented pre-training for table semantic parsing. *arXiv preprint arXiv:2009.13845*, 2020.
- Lu Zeng, Sree Hari Krishnan Parthasarathi, and Dilek Hakkani-Tur. N-best hypotheses reranking for text-to-sql systems. *arXiv preprint arXiv:2210.10668*, 2022.
- Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. *arXiv preprint arXiv:2210.03493*, 2022.
- Yiyun Zhao, Jiarong Jiang, Yiqun Hu, Wuwei Lan, Henry Zhu, Anuj Chauhan, Alexander Li, Lin Pan, Jun Wang, Chung-Wei Hang, et al. Importance of synthesizing high-quality data for text-to-sql parsing. *arXiv preprint arXiv:2212.08785*, 2022.

- Ruiqi Zhong, Tao Yu, and Dan Klein. Semantic evaluation for text-to-sql with distilled test suite. In *The 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2020.
- Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.

A Prompts

This section presents a comprehensive list of all the prompts utilized in the four modules of our proposed methodology on both the GPT-4 and CodeX models. The prompts used for each module are provided in detail to allow for easy replication and understanding of the approach. Additionally, we have also included the prompt we used for the few-shot and zero-shot implementations of our method.

For our few-shot examples used for the Non-Nested Complex and Nested Complex classes of queries, we used the NatSQL intermediate representations from the NatSQL Github repository ². The repository gives the intermediate representation for all queries in the training set of Spider.

²<https://github.com/ygan/NatSQL>

A.1 Zero-shot prompting

The prompt utilized for the zero-shot prompting scenario draws its inspiration from the work of Liu et al. [2023a], proposed for the ChatGPT. In figure 5, we demonstrate one example for the Zero-shot prompting used in our work.

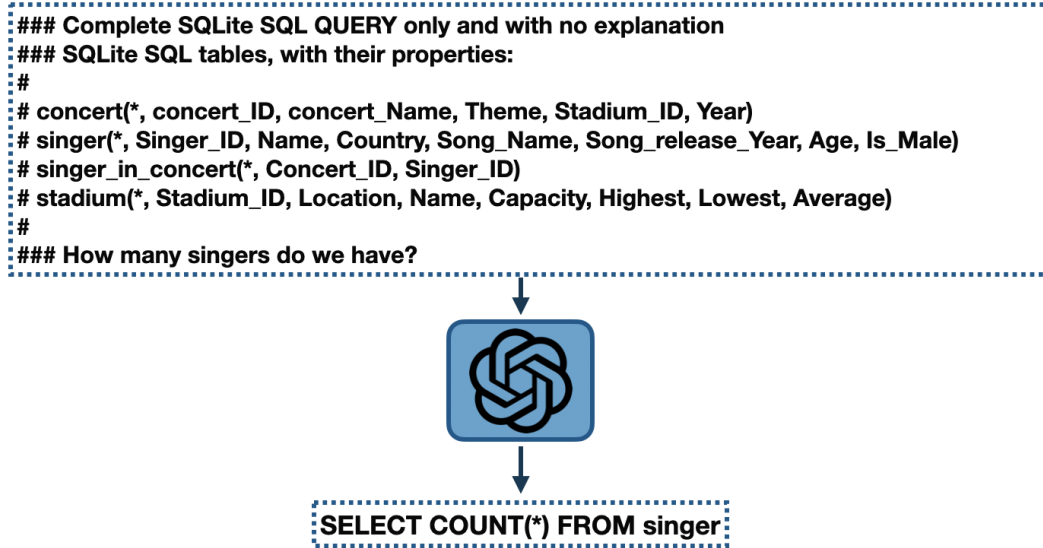


Figure 5: An example of Zero-shot prompting.

A.2 Few-shot prompting

Create SQL queries for the given questions.

Table advisor, columns = [*,s_ID,i_ID]
Table classroom, columns = [*,building,room_number,capacity]
Table course, columns = [*,course_id,title,dept_name,credits]
Table department, columns = [*,dept_name,building,budget]
Table instructor, columns = [*,ID,name,dept_name,salary]
Table prereq, columns = [*,course_id,prereq_id]
Table section, columns = [*,course_id,sec_id,semester,year,building,room_number,time_slot_id]
Table student, columns = [*,ID,name,dept_name,tot_cred]
Table takes, columns = [*,ID,course_id,sec_id,semester,year,grade]
Table teaches, columns = [*,ID,course_id,sec_id,semester,year]
Table time_slot, columns = [*,time_slot_id,day,start_hr,start_min,end_hr,end_min]

Q: "Find the buildings which have rooms with capacity more than 50."

SQL: **SELECT DISTINCT** building **FROM** classroom **WHERE** capacity > 50

Q: "Find the room number of the rooms which can sit 50 to 100 students and their buildings."

SQL: **SELECT** building , room_number **FROM** classroom **WHERE** capacity **BETWEEN** 50 **AND** 100

Q: "Give the name of the student in the History department with the most credits."

SQL: **SELECT** name **FROM** student **WHERE** dept_name = 'History' **ORDER BY** tot_cred **DESC LIMIT** 1

Q: "Find the total budgets of the Marketing or Finance department."

SQL: **SELECT** sum(budget) **FROM** department **WHERE** dept_name = 'Marketing' **OR** dept_name =

'Finance'

Q: "Find the department name of the instructor whose name contains 'Soisalon'."

SQL: **SELECT** dept_name **FROM** instructor **WHERE** name **LIKE** '%Soisalon%'

Q: "What is the name of the department with the most credits?"

SQL: **SELECT** dept_name **FROM** course **GROUP BY** dept_name **ORDER BY** sum(credits) **DESC LIMIT** 1

Q: "How many instructors teach a course in the Spring of 2010?"

SQL: **SELECT** COUNT (DISTINCT ID) **FROM** teaches **WHERE** semester = 'Spring' **AND** YEAR = 2010

Q: "Find the name of the students and their department names sorted by their total credits in ascending order."

SQL: **SELECT** name , dept_name **FROM** student **ORDER BY** tot_cred

Q: "Find the year which offers the largest number of courses."

SQL: **SELECT** YEAR **FROM** SECTION **GROUP BY** YEAR **ORDER BY** count(*) **DESC LIMIT** 1

Q: "What are the names and average salaries for departments with average salary higher than 42000?"

SQL: **SELECT** dept_name , AVG (salary) **FROM** instructor **GROUP BY** dept_name **HAVING** AVG (salary) > 42000

Q: "How many rooms in each building have a capacity of over 50?"

SQL: **SELECT** count(*) , building **FROM** classroom **WHERE** capacity > 50 **GROUP BY** building

Q: "Find the names of the top 3 departments that provide the largest amount of courses?"

SQL: **SELECT** dept_name **FROM** course **GROUP BY** dept_name **ORDER BY** count(*) **DESC LIMIT** 3

Q: "Find the maximum and average capacity among rooms in each building."

SQL: **SELECT** max(capacity) , avg(capacity) , building **FROM** classroom **GROUP BY** building

Q: "Find the title of the course that is offered by more than one department."

SQL: **SELECT** title **FROM** course **GROUP BY** title **HAVING** count(*) > 1

Q: "Find the total budgets of the Marketing or Finance department."

SQL: **SELECT** sum(budget) **FROM** department **WHERE** dept_name = 'Marketing' **OR** dept_name = 'Finance'

Q: "Find the name and building of the department with the highest budget."

SQL: **SELECT** dept_name , building **FROM** department **ORDER BY** budget **DESC LIMIT** 1

Q: "What is the name and building of the departments whose budget is more than the average budget?"

SQL: **SELECT** dept_name , building **FROM** department **WHERE** budget > (SELECT avg(budget) FROM department)

Q: "Find the total number of students and total number of instructors for each department."

SQL: **SELECT** count(DISTINCT T2.id) , count(DISTINCT T3.id) , T3.dept_name **FROM** department **AS** T1 **JOIN** student **AS** T2 **ON** T1.dept_name = T2.dept_name **JOIN** instructor **AS** T3 **ON** T1.dept_name = T3.dept_name **GROUP BY** T3.dept_name

Q: "Find the title of courses that have two prerequisites?"

SQL: **SELECT** T1.title **FROM** course **AS** T1 **JOIN** prereq **AS** T2 **ON** T1.course_id = T2.course_id **GROUP BY** T2.course_id **HAVING** count(*) = 2

Q: "Find the name of students who took any class in the years of 2009 and 2010."

SQL: `SELECT DISTINCT T1.name FROM student AS T1 JOIN takes AS T2 ON T1.id = T2.id WHERE T2.YEAR = 2009 OR T2.YEAR = 2010`

Q: "list in alphabetic order all course names and their instructors' names in year 2008."

SQL: `SELECT T1.title , T3.name FROM course AS T1 JOIN teaches AS T2 ON T1.course_id = T2.course_id JOIN instructor AS T3 ON T2.id = T3.id WHERE T2.YEAR = 2008 ORDER BY T1.title`

Q: "Find the title of courses that have two prerequisites?"

SQL: `SELECT T1.title FROM course AS T1 JOIN prereq AS T2 ON T1.course_id = T2.course_id GROUP BY T2.course_id HAVING count(*) = 2`

Q: "Find the name and building of the department with the highest budget."

SQL: `SELECT dept_name , building FROM department ORDER BY budget DESC LIMIT 1`

Q: "Find the title, credit, and department name of courses that have more than one prerequisites?"

SQL: `SELECT T1.title , T1.credits , T1.dept_name FROM course AS T1 JOIN prereq AS T2 ON T1.course_id = T2.course_id GROUP BY T2.course_id HAVING count(*) > 1`

Q: "Give the name and building of the departments with greater than average budget."

SQL: `SELECT dept_name , building FROM department WHERE budget > (SELECT avg(budget) FROM department)`

Q: "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010."

SQL: `SELECT id FROM teaches WHERE semester = 'Fall' AND YEAR = 2009 EXCEPT SELECT id FROM teaches WHERE semester = 'Spring' AND YEAR = 2010`

Q: "Find the name of the courses that do not have any prerequisite?"

SQL: `SELECT title FROM course WHERE course_id NOT IN (SELECT course_id FROM prereq)`

Q: "Find the salaries of all distinct instructors that are less than the largest salary."

SQL: `SELECT DISTINCT salary FROM instructor WHERE salary < (SELECT max(salary) FROM instructor)`

Q: "Find the names of students who have taken any course in the fall semester of year 2003."

SQL: `SELECT name FROM student WHERE id IN (SELECT id FROM takes WHERE semester = 'Fall' AND YEAR = 2003)`

Q: "Find the minimum salary for the departments whose average salary is above the average payment of all instructors."

SQL: `SELECT min(salary) , dept_name FROM instructor GROUP BY dept_name HAVING avg(salary) > (SELECT avg(salary) FROM instructor)`

Q: "What is the course title of the prerequisite of course Mobile Computing?"

SQL: `SELECT title FROM course WHERE course_id IN (SELECT T1.prereq_id FROM prereq AS T1 JOIN course AS T2 ON T1.course_id = T2.course_id WHERE T2.title = 'Mobile Computing')`

Q: "Give the title and credits for the course that is taught in the classroom with the greatest capacity."

SQL: `SELECT T3.title , T3.credits FROM classroom AS T1 JOIN SECTION AS T2 ON T1.building = T2.building AND T1.room_number = T2.room_number JOIN course AS T3 ON T2.course_id = T3.course_id WHERE T1.capacity = (SELECT max(capacity) FROM classroom)`

A.3 Schema linking prompt

Find the schema_links for generating SQL queries for each question based on the database schema and Foreign keys.

Table city, columns = [*,City_ID,Official_Name,Status,Area_km_2,Population,Census_Ranking]
Table competition_record, columns = [*,Competition_ID,Farm_ID,Rank]
Table farm, columns = [*,Farm_ID,Year>Total_Horses,Working_Horses,
Total_Cattle,Oxen,Bulls,Cows,Pigs,Sheep_and_Goats]
Table farm_competition, columns = [*,Competition_ID,Year,Theme,Host_city_ID,Hosts]
Foreign_keys = [farm_competition.Host_city_ID = city.City_ID,competition_record.Farm_ID =
farm.Farm_ID,competition_record.Competition_ID = farm_competition.Competition_ID]

Q: "Show the status of the city that has hosted the greatest number of competitions."

A: Let's think step by step. In the question "Show the status of the city that has hosted the greatest number of competitions.", we are asked:

"the status of the city" so we need column = [city.Status]

"greatest number of competitions" so we need column = [farm_competition.*]

Based on the columns and tables, we need these Foreign_keys = [farm_competition.Host_city_ID =
city.City_ID].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = []. So the
Schema_links are:

Schema_links: [city.Status,farm_competition.Host_city_ID = city.City_ID,farm_competition.*]

Table department, columns = [*,Department_ID,Name,Creation,Ranking,Budget_in_Billions,Num_Employees]

Table head, columns = [*,head_ID,name,born_state,age]

Table management, columns = [*,department_ID,head_ID,temporary_acting]

Foreign_keys = [management.head_ID = head.head_ID,management.department_ID = department.Department_ID]

Q: "How many heads of the departments are older than 56 ?"

A: Let's think step by step. In the question "How many heads of the departments are older than 56 ?",
we are asked:

"How many heads of the departments" so we need column = [head.*]

"older" so we need column = [head.age]

Based on the columns and tables, we need these Foreign_keys = [].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = [56]. So the
Schema_links are:

Schema_links: [head.*,head.age,56]

Table department, columns = [*,Department_ID,Name,Creation,Ranking,Budget_in_Billions,Num_Employees]

Table head, columns = [*,head_ID,name,born_state,age]

Table management, columns = [*,department_ID,head_ID,temporary_acting]

Foreign_keys = [management.head_ID = head.head_ID,management.department_ID = department.Department_ID]

Q: "what are the distinct creation years of the departments managed by a secretary born in state
'Alabama'?"

A: Let's think step by step. In the question "what are the distinct creation years of the departments
managed by a secretary born in state 'Alabama'?", we are asked:

"distinct creation years of the departments" so we need column = [department.Creation]

"departments managed by" so we need column = [management.department_ID]

"born in" so we need column = [head.born_state]

Based on the columns and tables, we need these Foreign_keys = [department.Department_ID =
management.department_ID,management.head_ID = head.head_ID].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = ['Alabama'].
So the Schema_links are:

Schema_links: [department.Creation,department.Department_ID = management.department_ID,
head.head_ID = management.head_ID,head.born_state,'Alabama']

Table Addresses, columns = [*,address_id,line_1,line_2,city,zip_postcode,state_province_county,country]

Table Candidate_Assessments, columns = [*,candidate_id,qualification,assessment_date,assessment_outcome_code]

Table Candidates, columns = [*,candidate_id,candidate_details]
Table Courses, columns = [*,course_id,course_name,course_description,other_details]
Table People, columns = [*,person_id,first_name,middle_name,last_name,cell_mobile_number,email_address,login_name,password]
Table People_Addresses, columns = [*,person_address_id,person_id,address_id,date_from,date_to]
Table Student_Course_Attendance, columns = [*,student_id,course_id,date_of_attendance]
Table Student_Course_Registrations, columns = [*,student_id,course_id,registration_date]
Table Students, columns = [*,student_id,student_details]
Foreign_keys = [Students.student_id = People.person_id,People_Addresses.address_id = Addresses.address_id,People_Addresses.person_id = People.person_id,Student_Course_Registrations.course_id = Courses.course_id,Student_Course_Registrations.student_id = Students.student_id,Student_Course_Attendance.student_id = Student_Course_Registrations.student_id,Student_Course_Attendance.course_id = Student_Course_Registrations.course_id,Candidates.candidate_id = People.person_id,Candidate_Assessments.candidate_id = Candidates.candidate_id]

Q: "List the id of students who never attends courses?"

A: Let's think step by step. In the question "List the id of students who never attends courses?", we are asked:

"id of students" so we need column = [Students.student_id]

"never attends courses" so we need column = [Student_Course_Attendance.student_id]

Based on the columns and tables, we need these Foreign_keys = [Students.student_id = Student_Course_Attendance.student_id].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = []. So the Schema_links are:

Schema_links: [Students.student_id = Student_Course_Attendance.student_id]

Table Country, columns = [*,id,name]

Table League, columns = [*,id,country_id,name]

Table Player, columns = [*,id,player_api_id,player_name,player_fifa_api_id,birthday,height,weight]

Table Player_Attributes, columns = [*,id,player_fifa_api_id,player_api_id,date,overall_rating,potential,preferred_foot,attacking_work_rate,defensive_work_rate,crossing,finishing,heading_accuracy,short_passing,volleys,dribbling,curve,free_kick_accuracy,long_passing,ball_control,acceleration,sprint_speed,agility,reactions,balance,shot_power,jumping,stamina,strength,long_shots,aggression,interceptions,positioning,vision,penalties,marking,standing_tackle,sliding_tackle,gk_diving,gk_handling,gk_kicking,gk_positioning,gk_reflexes]

Table Team, columns = [*,id,team_api_id,team_fifa_api_id,team_long_name,team_short_name]

Table Team_Attributes, columns = [*,id,team_fifa_api_id,team_api_id,date,buildUpPlaySpeed,buildUpPlaySpeedClass,buildUpPlayDribbling,buildUpPlayDribblingClass,buildUpPlayPassing,buildUpPlayPassingClass,buildUpPlayPositioningClass,chanceCreationPassing,chanceCreationPassingClass,chanceCreationCrossing,chanceCreationCrossingClass,chanceCreationShooting,chanceCreationShootingClass,chanceCreationPositioningClass,defencePressure,defencePressureClass,defenceAggression,defenceAggressionClass,defenceTeamWidth,defenceTeamWidthClass,defenceDefenderLineClass]

Table sqlite_sequence, columns = [*,name,seq]

Foreign_keys = [Player_Attributes.player_api_id = Player.player_api_id,

Player_Attributes.player_fifa_api_id = Player.player_fifa_api_id,

League.country_id = Country.id,Team_Attributes.team_api_id = Team.team_api_id,

Team_Attributes.team_fifa_api_id = Team.team_fifa_api_id]

Q: "List the names of all left-footed players who have overall rating between 85 and 90."

A: Let's think step by step. In the question "List the names of all left-footed players who have overall rating between 85 and 90.", we are asked:

"names of all left-footed players" so we need column = [Player.player_name,Player_Attributes.preferred_foot]

"players who have overall rating" so we need column = [Player_Attributes.overall_rating]

Based on the columns and tables, we need these Foreign_keys = [Player_Attributes.player_api_id = Player.player_api_id].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = [left,85,90]. So

the Schema_links are:

Schema_links: [Player.player_name,Player_Attributes.preferred_foot,Player_Attributes.overall_rating,Player_Attributes.player_api_id = Player.player_api_id,left,85,90]

Table advisor, columns = [*,s_ID,i_ID]

Table classroom, columns = [*,building,room_number,capacity]

Table course, columns = [*,course_id,title,dept_name,credits]

Table department, columns = [*,dept_name,building,budget]

Table instructor, columns = [*,ID,name,dept_name,salary]

Table prereq, columns = [*,course_id,prereq_id]

Table section, columns = [*,course_id,sec_id,semester,year,building,room_number,time_slot_id]

Table student, columns = [*,ID,name,dept_name,tot_cred]

Table takes, columns = [*,ID,course_id,sec_id,semester,year,grade]

Table teaches, columns = [*,ID,course_id,sec_id,semester,year]

Table time_slot, columns = [*,time_slot_id,day,start_hr,start_min,end_hr,end_min]

Foreign_keys = [course.dept_name = department.dept_name,instructor.dept_name = department.dept_name,section.building = classroom.building

,section.room_number = classroom.room_number

,section.course_id = course.course_id,teaches.ID = instructor.ID,teaches.course_id = section.course_id,teaches.sec_id = section.sec_id,

teaches.semester = section.semester,teaches.year = section.year,student.dept_name = department.dept_name,

takes.ID = student.ID,takes.course_id = section.course_id,

takes.sec_id = section.sec_id,takes.semester = section.semester,

takes.year = section.year,advisor.s_ID = student.ID,

advisor.i_ID = instructor.ID,prereq.prereq_id = course.course_id,prereq.course_id = course.course_id]

Q: "Give the title of the course offered in Chandler during the Fall of 2010."

A: Let's think step by step. In the question "Give the title of the course offered in Chandler during the Fall of 2010.", we are asked:

"title of the course" so we need column = [course.title]

"course offered in Chandler" so we need column = [SECTION.building]

"during the Fall" so we need column = [SECTION.semester]

"of 2010" so we need column = [SECTION.year]

Based on the columns and tables, we need these Foreign_keys = [course.course_id = SECTION.course_id].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = [Chandler,Fall,2010]. So the Schema_links are:

Schema_links: [course.title,course.course_id = SECTION.course_id,SECTION.building,SECTION.year,SECTION.semester,Chandler,Fall,2010]

Table advisor, columns = [*,s_ID,i_ID]

Table classroom, columns = [*,building,room_number,capacity]

Table course, columns = [*,course_id,title,dept_name,credits]

Table department, columns = [*,dept_name,building,budget]

Table instructor, columns = [*,ID,name,dept_name,salary]

Table prereq, columns = [*,course_id,prereq_id]

Table section, columns = [*,course_id,sec_id,semester,year,building,room_number,time_slot_id]

Table student, columns = [*,ID,name,dept_name,tot_cred]

Table takes, columns = [*,ID,course_id,sec_id,semester,year,grade]

Table teaches, columns = [*,ID,course_id,sec_id,semester,year]

Table time_slot, columns = [*,time_slot_id,day,start_hr,start_min,end_hr,end_min]

Foreign_keys = [course.dept_name = department.dept_name,instructor.dept_name = department.dept_name,

section.building = classroom.building,section.room_number = classroom.room_number,

section.course_id = course.course_id,teaches.ID = instructor.ID,teaches.course_id = section.course_id,

teaches.sec_id = section.sec_id,teaches.semester = section.semester,teaches.year = section.year,

student.dept_name = department.dept_name,takes.ID = student.ID,takes.course_id = sec-

tion.course_id,
 takes.sec_id = section.sec_id,takes.semester = section.semester,
 takes.year = section.year,advisor.s_ID = student.ID,advisor.i_ID = instructor.ID,
 prereq.prereq_id = course.course_id,prereq.course_id = course.course_id]
Q: "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010."

A: Let's think step by step. In the question "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010.", we are asked:

"id of instructors who taught " so we need column = [teaches.id]
 "taught a class in" so we need column = [teaches.semester,teaches.year]

Based on the columns and tables, we need these Foreign_keys = [].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = [Fall,2009,Spring,2010]. So the Schema_links are:

Schema_links: [teaches.id,teaches.semester,teaches.year,Fall,2009,Spring,2010]

Table Accounts, columns = [*,account_id,customer_id,date_account_opened,account_name,other_account_details]

Table Customers, columns = [*,customer_id,customer_first_name,customer_middle_initial,
 customer_last_name,gender,email_address,login_name,login_password,phone_number,
 town_city,state_county_province,country]

Table Financial_Transactions, columns = [*,transaction_id,account_id,invoice_number,transaction_type,
 transaction_date,transaction_amount,transaction_comment,other_transaction_details]

Table Invoice_Line_Items, columns = [*,order_item_id,invoice_number,product_id,product_title,product_quantity,
 product_price,derived_product_cost,derived_vat_payable,derived_total_cost]

Table Invoices, columns = [*,invoice_number,order_id,invoice_date]

Table Order_Items, columns = [*,order_item_id,order_id,product_id,product_quantity,other_order_item_details]

Table Orders, columns = [*,order_id,customer_id,date_order_placed,order_details]

Table Product_Categories, columns = [*,production_type_code,product_type_description,vat_rating]

Table Products, columns = [*,product_id,parent_product_id,production_type_code,
 unit_price,product_name,product_color,product_size]

Foreign_keys = [Orders.customer_id = Customers.customer_id,Invoices.order_id = Or-
 ders.order_id,Accounts.customer_id = Customers.customer_id,

Products.production_type_code = Product_Categories.production_type_code,Financial_Transactions.account_id
 = Accounts.account_id,Financial_Transactions.invoice_number = Invoices.invoice_number,

Order_Items.order_id = Orders.order_id,Order_Items.product_id = Products.product_id,

Invoice_Line_Items.product_id = Products.product_id,Invoice_Line_Items.invoice_number =
 Invoices.invoice_number,

Invoice_Line_Items.order_item_id = Order_Items.order_item_id]

Q: "Show the id, the date of account opened, the account name, and other account detail for all
 accounts."

A: Let's think step by step. In the question "Show the id, the date of account opened, the account
 name, and other account detail for all accounts.", we are asked:

"the id, the date of account opened, the account name, and other account detail for all accounts." so
 we need column = [Accounts.account_id,

Accounts.account_name,Accounts.other_account_details,Accounts.date_account_opened]

Based on the columns and tables, we need these Foreign_keys = [].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = []. So the
 Schema_links are:

Schema_links: [Accounts.account_id,Accounts.account_name,
 Accounts.other_account_details,Accounts.date_account_opened]

Table advisor, columns = [*,s_ID,i_ID]

Table classroom, columns = [*,building,room_number,capacity]

Table course, columns = [*,course_id,title,dept_name,credits]

Table department, columns = [*,dept_name,building,budget]

Table instructor, columns = [*,ID,name,dept_name,salary]

Table prereq, columns = [*,course_id,prereq_id]

Table section, columns = [*,course_id,sec_id,semester,year,building,room_number,time_slot_id]

Table student, columns = [*,ID,name,dept_name,tot_cred]

Table takes, columns = [*,ID,course_id,sec_id,semester,year,grade]

Table teaches, columns = [*,ID,course_id,sec_id,semester,year]

Table time_slot, columns = [*,time_slot_id,day,start_hr,start_min,end_hr,end_min]
Foreign_keys = [course.dept_name = department.dept_name,instructor.dept_name = department.dept_name,
section.building = classroom.building,section.room_number = classroom.room_number,
section.course_id = course.course_id,teaches.ID = instructor.ID,teaches.course_id = section.course_id,teaches.sec_id = section.sec_id,
teaches.semester = section.semester,teaches.year = section.year,student.dept_name = department.dept_name,takes.ID = student.ID,takes.course_id = section.course_id,
takes.sec_id = section.sec_id,takes.semester = section.semester,takes.year = section.year,advisor.s_ID = student.ID,
advisor.i_ID = instructor.ID,prereq.prereq_id = course.course_id,prereq.course_id = course.course_id]

Q: "Find the buildings which have rooms with capacity more than 50."

A: Let's think step by step. In the question "Find the buildings which have rooms with capacity more than 50.", we are asked:

"the buildings which have rooms" so we need column = [classroom.capacity]

"rooms with capacity" so we need column = [classroom.building]

Based on the columns and tables, we need these Foreign_keys = [].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = [50]. So the Schema_links are:

Schema_links: [classroom.building,classroom.capacity,50]

Table city, columns = [*,City_ID,Official_Name,Status,Area_km_2,Population,Census_Ranking]

Table competition_record, columns = [*,Competition_ID,Farm_ID,Rank]

Table farm, columns = [*,Farm_ID,Year,Total_Horses,Working_Horses,Total_Cattle,Oxen,Bulls,Cows,Pigs,Sheep_and_Goats]

Table farm_competition, columns = [*,Competition_ID,Year,Theme,Host_city_ID,Hosts]

Foreign_keys = [farm_competition.Host_city_ID = city.City_ID,competition_record.Farm_ID = farm.Farm_ID,competition_record.Competition_ID = farm_competition.Competition_ID]

Q: "Show the status shared by cities with population bigger than 1500 and smaller than 500."

A: Let's think step by step. In the question "Show the status shared by cities with population bigger than 1500 and smaller than 500.", we are asked:

"the status shared by cities" so we need column = [city.Status]

"cities with population" so we need column = [city.Population]

Based on the columns and tables, we need these Foreign_keys = [].

Based on the tables, columns, and Foreign_keys, The set of possible cell values are = [1500,500]. So the Schema_links are:

Schema_links: [city.Status,city.Population,1500,500]

A.4 Classification & decomposition prompt

For the given question, classify it as EASY, NON-NESTED, or NESTED based on nested queries and JOIN.

if need nested queries: predict NESTED

elif need JOIN and don't need nested queries: predict NON-NESTED

elif don't need JOIN and don't need nested queries: predict EASY

Table advisor, columns = [*,s_ID,i_ID]

Table classroom, columns = [*,building,room_number,capacity]

Table course, columns = [*,course_id,title,dept_name,credits]

Table department, columns = [*,dept_name,building,budget]

Table instructor, columns = [*,ID,name,dept_name,salary]

Table prereq, columns = [*,course_id,prereq_id]

Table section, columns = [*,course_id,sec_id,semester,year,building,room_number,time_slot_id]

Table student, columns = [*,ID,name,dept_name,tot_cred]

Table takes, columns = [*,ID,course_id,sec_id,semester,year,grade]

Table teaches, columns = [*,ID,course_id,sec_id,semester,year]

Table time_slot, columns = [*,time_slot_id,day,start_hr,start_min,end_hr,end_min]

Foreign_keys = [course.dept_name = department.dept_name,instructor.dept_name = department.dept_name,section.building = classroom.building,section.room_number = classroom.room_number,
 section.course_id = course.course_id,teaches.ID = instructor.ID,teaches.course_id = section.course_id,teaches.sec_id = section.sec_id,teaches.semester = section.semester,
 teaches.year = section.year,student.dept_name = department.dept_name,takes.ID = student.ID,takes.course_id = section.course_id,takes.sec_id = section.sec_id,takes.semester = section.semester,takes.year = section.year,advisor.s_ID = student.ID,
 advisor.i_ID = instructor.ID,prereq.prereq_id = course.course_id,prereq.course_id = course.course_id]

Q: "Find the buildings which have rooms with capacity more than 50."

schema_links: [classroom.building,classroom.capacity,50]

A: Let's think step by step. The SQL query for the question "Find the buildings which have rooms with capacity more than 50." needs these tables = [classroom], so we don't need JOIN.

Plus, it doesn't require nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = [""].

So, we don't need JOIN and don't need nested queries, then the the SQL query can be classified as "EASY".

Label: "EASY"

Q: "What are the names of all instructors who advise students in the math depart sorted by total credits of the student."

schema_links: [advisor.i_id = instructor.id,advisor.s_id = student.id,instructor.name, student.dept_name,student.tot_cred,math]

A: Let's think step by step. The SQL query for the question "What are the names of all instructors who advise students in the math depart sorted by total credits of the student." needs these tables = [advisor,instructor,student], so we need JOIN.

Plus, it doesn't need nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = [""].

So, we need JOIN and don't need nested queries, then the the SQL query can be classified as "NON-NESTED".

Label: "NON-NESTED"

Q: "How many courses that do not have prerequisite?"

schema_links: [course.*,course.course_id = prereq.course_id]

A: Let's think step by step. The SQL query for the question "How many courses that do not have prerequisite?" needs these tables = [course,prereq], so we need JOIN.

Plus, it requires nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = ["Which courses have prerequisite?"].

So, we need JOIN and need nested queries, then the the SQL query can be classified as "NESTED".

Label: "NESTED"

Q: "Find the room number of the rooms which can sit 50 to 100 students and their buildings."

schema_links: [classroom.building,classroom.room_number,classroom.capacity,50,100]

A: Let's think step by step. The SQL query for the question "Find the room number of the rooms which can sit 50 to 100 students and their buildings." needs these tables = [classroom], so we don't need JOIN.

Plus, it doesn't require nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = [""].

So, we don't need JOIN and don't need nested queries, then the the SQL query can be classified as "EASY".

Label: "EASY"

Q: "Find the title of course that is provided by both Statistics and Psychology departments."

schema_links: [course.title,course.dept_name,Statistics,Psychology]

A: Let's think step by step. The SQL query for the question "Find the title of course that is provided

by both Statistics and Psychology departments." needs these tables = [course], so we don't need JOIN.

Plus, it requires nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = ["Find the titles of courses that is provided by Psychology departments"]. So, we don't need JOIN and need nested queries, then the the SQL query can be classified as "NESTED".

Label: "NESTED"

Q: "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010."

schema_links: [teaches.id,teaches.semester,teaches.year,Fall,2009,Spring,2010]

A: Let's think step by step. The SQL query for the question "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010." needs these tables = [teaches], so we don't need JOIN.

Plus, it requires nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = ["Find the id of instructors who taught a class in Spring 2010"].

So, we don't need JOIN and need nested queries, then the the SQL query can be classified as "NESTED".

Label: "NESTED"

Q: "Find the name of the department that offers the highest total credits?"

schema_links: [course.dept_name,course.credits]

A: Let's think step by step. The SQL query for the question "Find the name of the department that offers the highest total credits?." needs these tables = [course], so we don't need JOIN.

Plus, it doesn't require nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = [""].

So, we don't need JOIN and don't need nested queries, then the the SQL query can be classified as "EASY".

Label: "EASY"

Q: "What is the name of the instructor who advises the student with the greatest number of total credits?"

schema_links: [advisor.i_id = instructor.id,advisor.s_id = student.id,instructor.name,student.tot_cred]

A: Let's think step by step. The SQL query for the question "What is the name of the instructor who advises the student with the greatest number of total credits?" needs these tables = [advisor,instructor,student], so we need JOIN.

Plus, it doesn't need nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = [""].

So, we need JOIN and don't need nested queries, then the the SQL query can be classified as "NON-NESTED".

Label: "NON-NESTED"

Q: "Find the total number of students and total number of instructors for each department."

schema_links: [department.dept_name = instructor.dept_name,student.id,student.dept_name = department.dept_name,instructor.id]

A: Let's think step by step. The SQL query for the question "Find the total number of students and total number of instructors for each department." needs these tables = [department,instructor,student], so we need JOIN.

Plus, it doesn't need nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need the answer to the questions = [""].

So, we need JOIN and don't need nested queries, then the the SQL query can be classified as "NON-NESTED".

Label: "NON-NESTED"

Q: "Give the name and building of the departments with greater than average budget."

schema_links: [department.budget,department.dept_name,department.building]

A: Let's think step by step. The SQL query for the question "Give the name and building of the departments with greater than average budget." needs these tables = [department], so we don't need JOIN.

Plus, it requires nested queries with (INTERSECT, UNION, EXCEPT, IN, NOT IN), and we need

the answer to the questions = ["What is the average budget of the departments"].
So, we don't need JOIN and need nested queries, then the the SQL query can be classified as "NESTED".

Label: "NESTED"

A.5 SQL generation

A.5.1 Easy Class

Use the the schema links to generate the SQL queries for each of the questions.

Table advisor, columns = [* ,s_ID,i_ID]

Table classroom, columns = [* ,building,room_number,capacity]

Table course, columns = [* ,course_id,title,dept_name,credits]

Table department, columns = [* ,dept_name,building,budget]

Table instructor, columns = [* ,ID,name,dept_name,salary]

Table prereq, columns = [* ,course_id,prereq_id]

Table section, columns = [* ,course_id,sec_id,semester,year,building,room_number,time_slot_id]

Table student, columns = [* ,ID,name,dept_name,tot_cred]

Table takes, columns = [* ,ID,course_id,sec_id,semester,year,grade]

Table teaches, columns = [* ,ID,course_id,sec_id,semester,year]

Table time_slot, columns = [* ,time_slot_id,day,start_hr,start_min,end_hr,end_min]

Q: "Find the buildings which have rooms with capacity more than 50."

Schema_links: [classroom.building,classroom.capacity,50]

SQL: **SELECT DISTINCT** building **FROM** classroom **WHERE** capacity > 50

Q: "Find the room number of the rooms which can sit 50 to 100 students and their buildings."

Schema_links: [classroom.building,classroom.room_number,classroom.capacity,50,100]

SQL: **SELECT** building , room_number **FROM** classroom **WHERE** capacity **BETWEEN** 50 **AND** 100

Q: "Give the name of the student in the History department with the most credits."

Schema_links: [student.name,student.dept_name,student.tot_cred,History]

SQL: **SELECT** name **FROM** student **WHERE** dept_name = 'History' **ORDER BY** tot_cred **DESC** **LIMIT** 1

Q: "Find the total budgets of the Marketing or Finance department."

Schema_links: [department.budget,department.dept_name,Marketing,Finance]

SQL: **SELECT** sum(budget) **FROM** department **WHERE** dept_name = 'Marketing' **OR** dept_name = 'Finance'

Q: "Find the department name of the instructor whose name contains 'Soisalon'."

Schema_links: [instructor.dept_name,instructor.name,Soisalon]

SQL: **SELECT** dept_name **FROM** instructor **WHERE** name **LIKE** '%Soisalon%'

Q: "What is the name of the department with the most credits?"

Schema_links: [course.dept_name,course.credits]

SQL: **SELECT** dept_name **FROM** course **GROUP BY** dept_name **ORDER BY** sum(credits) **DESC** **LIMIT** 1

Q: "How many instructors teach a course in the Spring of 2010?"

Schema_links: [teaches.ID,teaches.semester,teaches.YEAR,Spring,2010]

SQL: **SELECT** COUNT (DISTINCT ID) **FROM** teaches **WHERE** semester = 'Spring' **AND** YEAR = 2010

Q: "Find the name of the students and their department names sorted by their total credits in ascending order."

Schema_links: [student.name,student.dept_name,student.tot_cred]

SQL: `SELECT name , dept_name FROM student ORDER BY tot_cred`

Q: "Find the year which offers the largest number of courses."

Schema_links: [SECTION.YEAR,SECTION.*]

SQL: `SELECT YEAR FROM SECTION GROUP BY YEAR ORDER BY count(*) DESC LIMIT 1`

Q: "What are the names and average salaries for departments with average salary higher than 42000?"

Schema_links: [instructor.dept_name,instructor.salary,42000]

SQL: `SELECT dept_name , AVG (salary) FROM instructor GROUP BY dept_name HAVING AVG (salary) > 42000`

Q: "How many rooms in each building have a capacity of over 50?"

Schema_links: [classroom.*,classroom.building,classroom.capacity,50]

SQL: `SELECT count(*) , building FROM classroom WHERE capacity > 50 GROUP BY building`

Q: "Find the names of the top 3 departments that provide the largest amount of courses?"

Schema_links: [course.dept_name,course.*]

SQL: `SELECT dept_name FROM course GROUP BY dept_name ORDER BY count(*) DESC LIMIT 3`

Q: "Find the maximum and average capacity among rooms in each building."

Schema_links: [classroom.building,classroom.capacity]

SQL: `SELECT max(capacity) , avg(capacity) , building FROM classroom GROUP BY building`

Q: "Find the title of the course that is offered by more than one department."

Schema_links: [course.title]

SQL: `SELECT title FROM course GROUP BY title HAVING count(*) > 1`

A.5.2 Non-Nested Complex

Use the the schema links and Intermediate_representation to generate the SQL queries for each of the questions.

Table advisor, columns = [*,s_ID,i_ID]

Table classroom, columns = [*,building,room_number,capacity]

Table course, columns = [*,course_id,title,dept_name,credits]

Table department, columns = [*,dept_name,building,budget]

Table instructor, columns = [*,ID,name,dept_name,salary]

Table prereq, columns = [*,course_id,prereq_id]

Table section, columns = [*,course_id,sec_id,semester,year,building,room_number,time_slot_id]

Table student, columns = [*,ID,name,dept_name,tot_cred]

Table takes, columns = [*,ID,course_id,sec_id,semester,year,grade]

Table teaches, columns = [*,ID,course_id,sec_id,semester,year]

Table time_slot, columns = [*,time_slot_id,day,start_hr,start_min,end_hr,end_min]

Foreign_keys = [course.dept_name = department.dept_name,instructor.dept_name = department.dept_name,section.building = classroom.building,

section.room_number = classroom.room_number,section.course_id = course.course_id,teaches.ID = instructor.ID,teaches.course_id = section.course_id,

teaches.sec_id = section.sec_id,teaches.semester = section.semester,teaches.year = section.year,

student.dept_name = department.dept_name,takes.ID = student.ID,takes.course_id = section.course_id,takes.sec_id = section.sec_id,takes.semester = section.semester,

takes.year = section.year,advisor.s_ID = student.ID,advisor.i_ID = instructor.ID,prereq.prereq_id = course.course_id,

prereq.course_id = course.course_id]

Q: "Find the total budgets of the Marketing or Finance department."

Schema_links: [department.budget,department.dept_name,Marketing,Finance]

A: Let's think step by step. For creating the SQL for the given question, we need to join these tables = [].

First, create an intermediate representation, then use it to construct the SQL query.

Intermediate_representation: select sum(department.budget) from department where department.dept_name = "Marketing" or department.dept_name = "Finance"

SQL: SELECT sum(budget) FROM department WHERE dept_name = 'Marketing' OR dept_name = 'Finance'

Q: "Find the name and building of the department with the highest budget."

Schema_links: [department.budget,department.dept_name,department.building]

A: Let's think step by step. For creating the SQL for the given question, we need to join these tables = [].

First, create an intermediate representation, then use it to construct the SQL query.

Intermediate_representation: select department.dept_name , department.building from department order by department.budget desc limit 1

SQL: SELECT dept_name , building FROM department ORDER BY budget DESC LIMIT 1

Q: "What is the name and building of the departments whose budget is more than the average budget?"

Schema_links: [department.budget,department.dept_name,department.building]

A: Let's think step by step. For creating the SQL for the given question, we need to join these tables = [].

First, create an intermediate representation, then use it to construct the SQL query.

Intermediate_representation: select department.dept_name , department.building from department where @.@ > avg (department.budget)

SQL: SELECT dept_name , building FROM department WHERE budget > (SELECT avg(budget) FROM department)

Q: "Find the total number of students and total number of instructors for each department."

Schema_links: [department.dept_name = student.dept_name,student.id,department.dept_name = instructor.dept_name,instructor.id]

A: Let's think step by step. For creating the SQL for the given question, we need to join these tables = [department,student,instructor].

First, create an intermediate representation, then use it to construct the SQL query.

Intermediate_representation: "select count(distinct student.ID) , count(distinct instructor.ID) , department.dept_name from department group by instructor.dept_name

SQL: SELECT count(DISTINCT T2.id) , count(DISTINCT T3.id) , T3.dept_name FROM department AS T1 JOIN student AS T2 ON T1.dept_name = T2.dept_name JOIN instructor AS T3 ON T1.dept_name = T3.dept_name GROUP BY T3.dept_name

Q: "Find the title of courses that have two prerequisites?"

Schema_links: [course.title,course.course_id = prereq.course_id]

A: Let's think step by step. For creating the SQL for the given question, we need to join these tables = [course,prereq].

First, create an intermediate representation, then use it to construct the SQL query.

Intermediate_representation: select course.title from course where count (prereq.*) = 2 group by prereq.course_id

SQL: SELECT T1.title FROM course AS T1 JOIN prereq AS T2 ON T1.course_id = T2.course_id GROUP BY T2.course_id HAVING count(*) = 2

Q: "Find the name of students who took any class in the years of 2009 and 2010."

Schema_links: [student.name,student.id = takes.id,takes.YEAR,2009,2010]

A: Let's think step by step. For creating the SQL for the given question, we need to join these tables = [student,takes].

First, create an intermediate representation, then use it to construct the SQL query.

Intermediate_representation: select distinct student.name from student where takes.year = 2009 or takes.year = 2010

SQL: SELECT DISTINCT T1.name FROM student AS T1 JOIN takes AS T2 ON T1.id = T2.id WHERE T2.YEAR = 2009 OR T2.YEAR = 2010

Q: "list in alphabetic order all course names and their instructors' names in year 2008."

Schema_links: [course.title,course.course_id = teaches.course_id,teaches.id = instructor.id,instructor.name,teaches.year,2008]

A: Let's think step by step. For creating the SQL for the given question, we need to join these tables = [course,teaches,instructor].

First, create an intermediate representation, then use it to construct the SQL query.

Intermediate_representation: select course.title , instructor.name from course where teaches.year = 2008 order by course.title asc

SQL: SELECT T1.title , T3.name FROM course AS T1 JOIN teaches AS T2 ON T1.course_id = T2.course_id JOIN instructor AS T3 ON T2.id = T3.id WHERE T2.YEAR = 2008 ORDER BY T1.title

A.5.3 Nested Complex

Use the intermediate representation and the schema links to generate the SQL queries for each of the questions.

Table advisor, columns = [*,s_ID,i_ID]

Table classroom, columns = [*,building,room_number,capacity]

Table course, columns = [*,course_id,title,dept_name,credits]

Table department, columns = [*,dept_name,building,budget]

Table instructor, columns = [*,ID,name,dept_name,salary]

Table prereq, columns = [*,course_id,prereq_id]

Table section, columns = [*,course_id,sec_id,semester,year,building,room_number,time_slot_id]

Table student, columns = [*,ID,name,dept_name,tot_cred]

Table takes, columns = [*,ID,course_id,sec_id,semester,year,grade]

Table teaches, columns = [*,ID,course_id,sec_id,semester,year]

Table time_slot, columns = [*,time_slot_id,day,start_hr,start_min,end_hr,end_min]

Foreign_keys = [course.dept_name = department.dept_name,instructor.dept_name = department.dept_name,section.building = classroom.building,section.room_number = classroom.room_number,

section.course_id = course.course_id,teaches.ID = instructor.ID,teaches.course_id = section.course_id,teaches.sec_id = section.sec_id,teaches.semester = section.semester,teaches.year = section.year,student.dept_name = department.dept_name,takes.ID = student.ID,takes.course_id = section.course_id,

takes.sec_id = section.sec_id,takes.semester = section.semester,takes.year = section.year,advisor.s_ID = student.ID,advisor.i_ID = instructor.ID,prereq.prereq_id = course.course_id,prereq.course_id = course.course_id]

Q: "Find the title of courses that have two prerequisites?"

Schema_links: [course.title,course.course_id = prereq.course_id]

A: Let's think step by step. "Find the title of courses that have two prerequisites?" can be solved by knowing the answer to the following sub-question "What are the titles for courses with two prerequisites?".

The SQL query for the sub-question "What are the titles for courses with two prerequisites?" is SELECT T1.title FROM course AS T1 JOIN prereq AS T2 ON T1.course_id = T2.course_id GROUP BY T2.course_id HAVING count(*) = 2

So, the answer to the question "Find the title of courses that have two prerequisites?" is =

Intermediate_representation: select course.title from course where count (prereq.*) = 2 group by prereq.course_id

SQL: SELECT T1.title FROM course AS T1 JOIN prereq AS T2 ON T1.course_id = T2.course_id GROUP BY T2.course_id HAVING count(*) = 2

Q: "Find the name and building of the department with the highest budget."

Schema_links: [department.dept_name,department.building,department.budget]

A: Let's think step by step. "Find the name and building of the department with the highest budget." can be solved by knowing the answer to the following sub-question "What is the department name and corresponding building for the department with the greatest budget?".

The SQL query for the sub-question "What is the department name and corresponding building for the department with the greatest budget?" is `SELECT dept_name , building FROM department ORDER BY budget DESC LIMIT 1`

So, the answer to the question "Find the name and building of the department with the highest budget." is =

Intermediate_representation: select department.dept_name , department.building from department order by department.budget desc limit 1

SQL: `SELECT dept_name , building FROM department ORDER BY budget DESC LIMIT 1`

Q: "Find the title, credit, and department name of courses that have more than one prerequisites?"

Schema_links: [course.title,course.credits,course.dept_name,course.course_id = prereq.course_id]

A: Let's think step by step. "Find the title, credit, and department name of courses that have more than one prerequisites?" can be solved by knowing the answer to the following sub-question "What is the title, credit value, and department name for courses with more than one prerequisite?".

The SQL query for the sub-question "What is the title, credit value, and department name for courses with more than one prerequisite?" is `SELECT T1.title , T1.credits , T1.dept_name FROM course AS T1 JOIN prereq AS T2 ON T1.course_id = T2.course_id GROUP BY T2.course_id HAVING count(*) > 1`

So, the answer to the question "Find the name and building of the department with the highest budget." is =

Intermediate_representation: select course.title , course.credits , course.dept_name from course where count (prereq.*) > 1 group by prereq.course_id

SQL: `SELECT T1.title , T1.credits , T1.dept_name FROM course AS T1 JOIN prereq AS T2 ON T1.course_id = T2.course_id GROUP BY T2.course_id HAVING count(*) > 1`

Q: "Give the name and building of the departments with greater than average budget."

Schema_links: [department.dept_name,department.building,department.budget]

A: Let's think step by step. "Give the name and building of the departments with greater than average budget." can be solved by knowing the answer to the following sub-question "What is the average budget of departments?".

The SQL query for the sub-question "What is the average budget of departments?" is `SELECT avg(budget) FROM department`

So, the answer to the question "Give the name and building of the departments with greater than average budget." is =

Intermediate_representation: select department.dept_name , department.building from department where @.@ > avg (department.budget)

SQL: `SELECT dept_name , building FROM department WHERE budget > (SELECT avg(budget) FROM department)`

Q: "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010."

Schema_links: [teaches.id,teaches.semester,teaches.YEAR,Fall,2009,Spring,2010]

A: Let's think step by step. "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010." can be solved by knowing the answer to the following sub-question "Find the id of instructors who taught a class in Spring 2010".

The SQL query for the sub-question "Find the id of instructors who taught a class in Spring 2010" is `SELECT id FROM teaches WHERE semester = 'Spring' AND YEAR = 2010`

So, the answer to the question "Find the id of instructors who taught a class in Fall 2009 but not in Spring 2010." is =

Intermediate_representation: select teaches.ID from teaches where teaches.semester = "Fall" and teaches.year = 2009 and teaches.semester != "Spring" and teaches.year = 2010

SQL: `SELECT id FROM teaches WHERE semester = 'Fall' AND YEAR = 2009 EXCEPT SELECT`

id FROM teaches WHERE semester = 'Spring' AND YEAR = 2010

Q: "Find the name of the courses that do not have any prerequisite?"

Schema_links: [course.title, course.course_id]

A: Let's think step by step. "Find the name of the courses that do not have any prerequisite?" can be solved by knowing the answer to the following sub-question "What are the courses that have any prerequisite?".

The SQL query for the sub-question "What are the courses that have any prerequisite?" is SELECT course_id FROM prereq

So, the answer to the question "Find the name of the courses that do not have any prerequisite?" is =

Intermediate_representation: select course.title from course where @.@ not in prereq.course_id

SQL: SELECT title FROM course WHERE course_id NOT IN (SELECT course_id FROM prereq)

Q: "Find the salaries of all distinct instructors that are less than the largest salary."

Schema_links: [instructor.salary]

A: Let's think step by step. "Find the salaries of all distinct instructors that are less than the largest salary." can be solved by knowing the answer to the following sub-question "What is the largest salary of instructors".

The SQL query for the sub-question "What is the largest salary of instructors" is SELECT max(salary) FROM instructor

So, the answer to the question "Find the salaries of all distinct instructors that are less than the largest salary." is =

Intermediate_representation: select distinct instructor.salary from instructor where @.@ < max (instructor.salary)

SQL: SELECT DISTINCT salary FROM instructor WHERE salary < (SELECT max(salary) FROM instructor)

Q: "Find the names of students who have taken any course in the fall semester of year 2003."

Schema_links: [student.id, student.name, takes.id, takes.semester, fall, 2003]

A: Let's think step by step. "Find the names of students who have taken any course in the fall semester of year 2003." can be solved by knowing the answer to the following sub-question "Find the students who have taken any course in the fall semester of year 2003."

The SQL query for the sub-question "Find the students who have taken any course in the fall semester of year 2003." is SELECT id FROM takes WHERE semester = 'Fall' AND YEAR = 2003

So, the answer to the question "Find the names of students who have taken any course in the fall semester of year 2003." is =

Intermediate_representation: select student.name from student where takes.semester = "Fall" and takes.year = 2003

SQL: SELECT name FROM student WHERE id IN (SELECT id FROM takes WHERE semester = 'Fall' AND YEAR = 2003)

Q: "Find the minimum salary for the departments whose average salary is above the average payment of all instructors."

Schema_links: [instructor.salary, instructor.dept_name]

A: Let's think step by step. "Find the minimum salary for the departments whose average salary is above the average payment of all instructors." can be solved by knowing the answer to the following sub-question "What is the average payment of all instructors."

The SQL query for the sub-question "What is the average payment of all instructors." is SELECT avg(salary) FROM instructor

So, the answer to the question "Find the minimum salary for the departments whose average salary is above the average payment of all instructors." is =

Intermediate_representation: select min(instructor.salary) , instructor.dept_name from instructor where avg (instructor.salary) > avg (instructor.salary) group by instructor.dept_name

SQL: SELECT min(salary) , dept_name FROM instructor GROUP BY dept_name HAVING avg(salary) > (SELECT avg(salary) FROM instructor)

Q: "What is the course title of the prerequisite of course Mobile Computing?"

Schema_links: [course.title,course.course_id = prereq.course_id,prereq.prereq_id,course.title,Mobile Computing]

A: Let's think step by step. "What is the course title of the prerequisite of course Mobile Computing?" can be solved by knowing the answer to the following sub-question "What are the ids of the prerequisite of course Mobile Computing?".

The SQL query for the sub-question "What are the ids of the prerequisite of course Mobile Computing?" is `SSELECT T1.prereq_id FROM prereq AS T1 JOIN course AS T2 ON T1.course_id = T2.course_id WHERE T2.title = 'Mobile Computing'`

So, the answer to the question "What is the course title of the prerequisite of course Mobile Computing?" is =

Intermediate_representation: select course.title from course where @.@ in prereq.* and course.title = "Mobile Computing"

SQL: `SELECT title FROM course WHERE course_id IN (SELECT T1.prereq_id FROM prereq AS T1 JOIN course AS T2 ON T1.course_id = T2.course_id WHERE T2.title = 'Mobile Computing')`

Q: "Give the title and credits for the course that is taught in the classroom with the greatest capacity."

Schema_links: [classroom.capacity,classroom.building = SECTION.building,classroom.room_number = SECTION.room_number,course.title,course.credits,course.course_id = SECTION.course_id]

A: Let's think step by step. "Give the title and credits for the course that is taught in the classroom with the greatest capacity." can be solved by knowing the answer to the following sub-question "What is the capacity of the largest room?".

The SQL query for the sub-question "What is the capacity of the largest room?" is `(SELECT max(capacity) FROM classroom)`

So, the answer to the question "Give the title and credits for the course that is taught in the classroom with the greatest capacity." is =

Intermediate_representation: select course.title , course.credits from classroom order by classroom.capacity desc limit 1"

SQL: `SELECT T3.title , T3.credits FROM classroom AS T1 JOIN SECTION AS T2 ON T1.building = T2.building AND T1.room_number = T2.room_number JOIN course AS T3 ON T2.course_id = T3.course_id WHERE T1.capacity = (SELECT max(capacity) FROM classroom)`

A.6 Self-correction prompts

A.6.1 Generic self-correction prompt

The Generic self-correction prompt was implemented in a zero-shot setting, where all queries were assumed to be "Buggy SQL". An example of this prompt is illustrated in Figure 6.

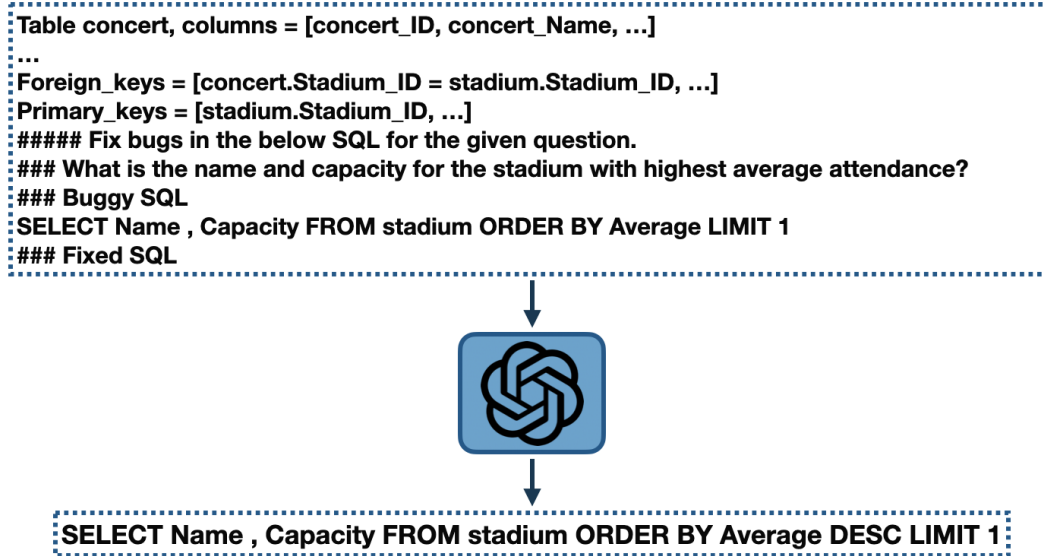


Figure 6: An example of Generic self-correction prompt.

A.6.2 Gentle self-correction prompt

The Gentle self-correction prompt was implemented in a zero-shot setting. For this self-correction prompt we don't have the assumption of being Buggy and we included some instructions for fixing the SQL queries. An example of this prompt is demonstrated in Figure 7.

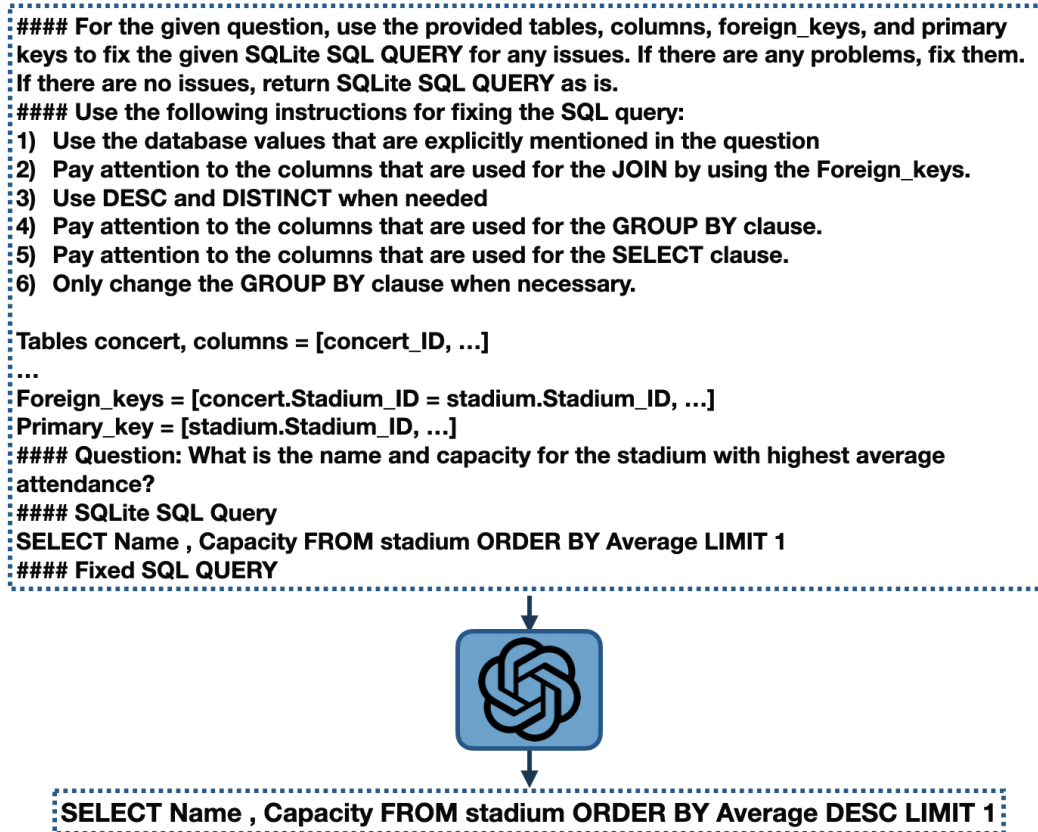


Figure 7: An example of Gentle self-correction prompt.